

EXPERIMENT 5:

Aim

To write a Python program that solves the Missionaries and Cannibals problem using the Breadth-First Search (BFS) algorithm.

Objective

To represent the state of missionaries, cannibals, and the boat on both sides of the river.

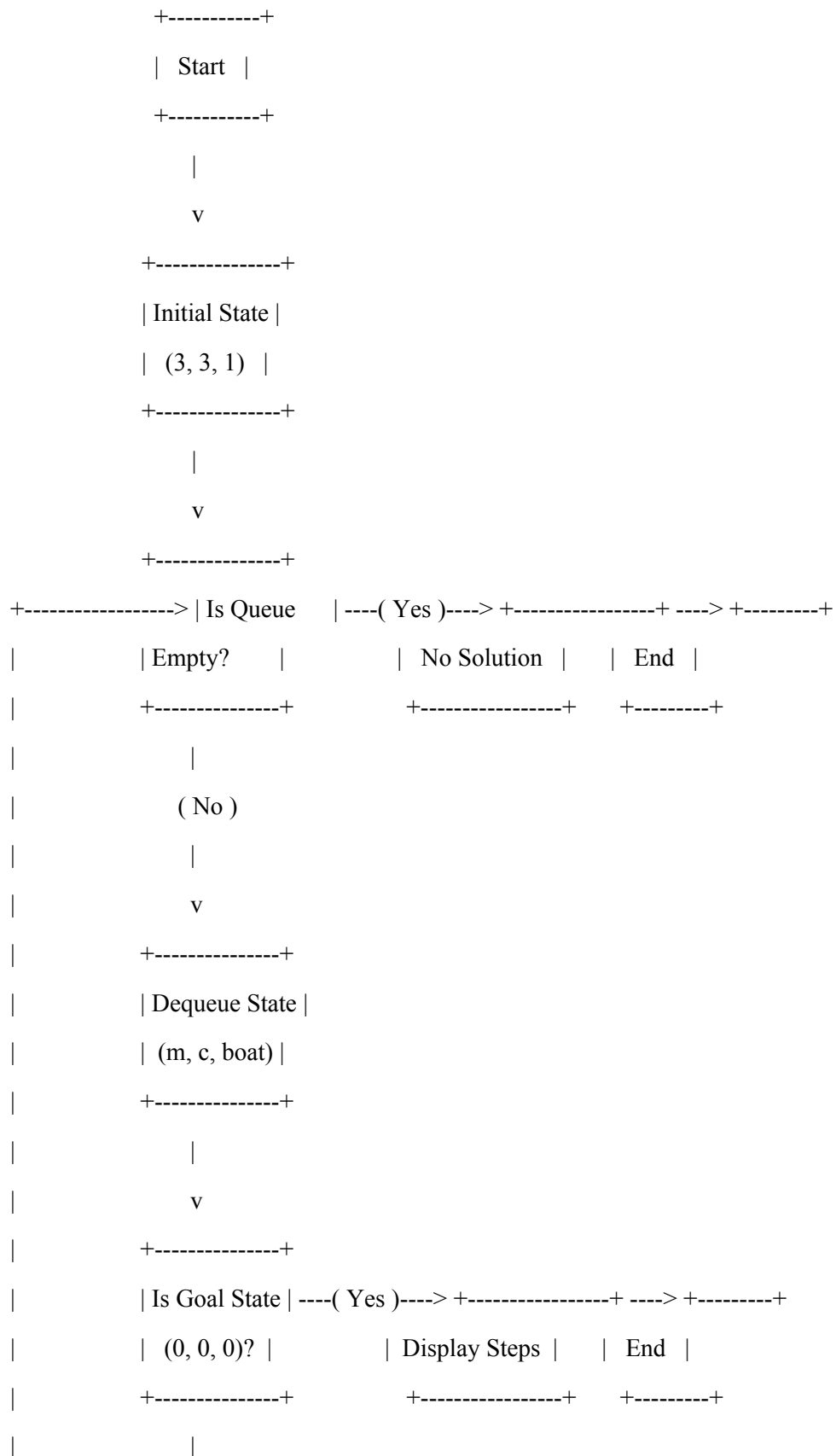
To enforce safety constraints so that cannibals never outnumber missionaries on either bank.

To find and display the step-by-step sequence of boat trips required to safely transport everyone across the river.

Algorithm

1. Define a state as a tuple (m, c, boat), where m is missionaries on the left bank, c is cannibals on the left bank, and boat is 1 (left bank) or 0 (right bank).
2. Initialize an open_list (queue) with the start state (3, 3, 1) and an empty path, along with a visited set.
3. Loop while the queue is not empty:
4. Dequeue the current state (m, c, boat) and its path.
5. If state matches the goal (0, 0, 0), return the solution path.
6. Check if the state is safe: on both banks, missionaries must either be 0 or greater than or equal to cannibals ($m \geq c$ and $(3 - m) \geq (3 - c)$).
7. If valid and not visited, mark the state as visited.
8. Generate possible moves for 1 or 2 people crossing in the boat: (1,0), (2,0), (0,1), (0,2), (1,1).
9. Calculate next state based on boat direction (subtract from left bank if moving left to right, add if moving right to left).
10. Enqueue all valid unvisited next states into the queue.

Flowchart



```

|                ( No )
|                |
|                v
|      +-----+
|      | Is State Safe | ----( No )-----+
|      | & Unvisited? |                |
|      +-----+                |
|                |                |
|                ( Yes )                |
|                |                |
|                v                |
|      +-----+                |
|      | Generate Next |                |
|      | Valid Moves  |                |
|      +-----+                |
|                |                |
|                v                |
|      +-----+                |
+-----+ Enqueue New  | <-----+
|   States   |
+-----+

```

Code

```
from collections import deque
```

```
def is_safe(m, c):
```

```
    if m < 0 or c < 0 or m > 3 or c > 3:
```

```
        return False
```

```
    if m > 0 and m < c:
```

```
        return False
```

```
    if (3 - m) > 0 and (3 - m) < (3 - c):
```

```
        return False
```

```
    return True
```

```

def solve_missionaries_cannibals():
    start = (3, 3, 1)
    goal = (0, 0, 0)
    queue = deque([(start, [])])
    visited = set()
    moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]
    while queue:
        (m, c, boat), path = queue.popleft()

        if (m, c, boat) == goal:
            return path + [(m, c, boat)]

        if (m, c, boat) in visited:
            continue
        visited.add((m, c, boat))
        current_path = path + [(m, c, boat)]
        for dm, dc in moves:
            if boat == 1:
                next_m, next_c, next_boat = m - dm, c - dc, 0
            else:
                next_m, next_c, next_boat = m + dm, c + dc, 1
            if is_safe(next_m, next_c):
                queue.append(((next_m, next_c, next_boat), current_path))
    return None

if name == "main":
    print("Solving Missionaries and Cannibals Problem...\n")
    solution = solve_missionaries_cannibals()
    if solution:
        print("Steps to transport everyone safely:")
        for step, (m, c, boat) in enumerate(solution):
            b_pos = "Left Bank" if boat == 1 else "Right Bank"

```

```
        print(f"Step {step}: Left Bank -> Missionaries: {m}, Cannibals: {c} | Boat: {b_pos}")
    else:
        print("No solution exists.")
```

Output

Solving Missionaries and Cannibals Problem...

Steps to transport everyone safely:

Step 0: Left Bank -> Missionaries: 3, Cannibals: 3 | Boat: Left Bank
Step 1: Left Bank -> Missionaries: 3, Cannibals: 1 | Boat: Right Bank
Step 2: Left Bank -> Missionaries: 3, Cannibals: 2 | Boat: Left Bank
Step 3: Left Bank -> Missionaries: 3, Cannibals: 0 | Boat: Right Bank
Step 4: Left Bank -> Missionaries: 3, Cannibals: 1 | Boat: Left Bank
Step 5: Left Bank -> Missionaries: 1, Cannibals: 1 | Boat: Right Bank
Step 6: Left Bank -> Missionaries: 2, Cannibals: 2 | Boat: Left Bank
Step 7: Left Bank -> Missionaries: 0, Cannibals: 2 | Boat: Right Bank
Step 8: Left Bank -> Missionaries: 0, Cannibals: 3 | Boat: Left Bank
Step 9: Left Bank -> Missionaries: 0, Cannibals: 1 | Boat: Right Bank
Step 10: Left Bank -> Missionaries: 0, Cannibals: 2 | Boat: Left Bank
Step 11: Left Bank -> Missionaries: 0, Cannibals: 0 | Boat: Right Bank

Result

The Missionaries and Cannibals program was executed successfully. Using the Breadth-First Search (BFS) algorithm, it identified the valid sequence of transitions to move all 3 missionaries and 3 cannibals across the river without violating safety conditions.

Conclusion

The Missionaries and Cannibals problem is effectively solved using graph search techniques like BFS. By pruning states that violate boundary or safety constraints, the algorithm efficiently finds the shortest sequence of boat crossings.